Paper 4: Structural Communication Safety Boundary for Deterministic O-RAN Authorization

David Kypuros
Red Hat
dkypuros@redhat.com

2 June 2026

Abstract

Paper 4 in this five-paper series describes the structural communication boundary used by the O-RAN Agent Harness to contain nondeterminism in Cloud RAN remediation. The harness does not let an LLM generate raw configuration, shell commands, direct REST calls, or live hardware mutations. Its first decision point is deterministic: a taxonomy-backed router classifies the fault and selects the service route, infrastructure route, or ambiguous class before any LLM is consulted. The LLM is only a support mechanism for ambiguous cases. Every proposed action must then pass a schema gate, a five-subcheck policy guardrail, and human co-authorization before it can be represented as a standards-shaped TMF688, TMF921, or O2 IMS handoff. The contribution of this paper is the structural communication boundary: nondeterministic reasoning can assist diagnosis, but execution authority is carried only by validated structures, policy results, audit records, and operator approval.

1 Introduction

The integration of agentic AI into telecom operations represents a fundamental shift in how networks are managed. A prior multivendor diagnostic publication demonstrated the efficacy of agents in root-cause localization across timing, RAN, and platform domains [2]. However, diagnosis is fundamentally an observation problem; remediation is a mutation problem. When an agent transitions from analyzing a fault payload to proposing a concrete physical change—such as a host reboot, firmware update, or driver swap—the tolerance for hallucination drops to zero.

The operational problem is not merely identifying the correct action, but safely translating a fault-driven remediation proposal into a physical or service-affecting change without allowing AI-generated text to become execution authority. In a multivendor O-RAN deployment, the executing environment is governed by strict standards [7, 10] and controlled by rigid operator contracts [6, 13]. Exposing these critical execution surfaces directly to an LLM is both technologically unsafe and operationally unacceptable.

This paper outlines the O-RAN Agent Harness's solution to this problem: structural demarcation. Section 2 explains the boundary between cognitive assistance and deterministic execution. Section 3 details the schema-driven containment strategy and the deterministic taxonomy/router. Section 4 describes the schema gate, five-subcheck policy guardrail, and human co-authorization chain. Section 5 maps the output to standards-shaped TM Forum and O-RAN envelopes. Section 7 contextualizes this approach within the broader telecom landscape. Section 8 states the v0 evidence boundary, and Section 9 concludes.

2 The Principle of Structural Demarcation

The Agent Harness architecture enforces a strict separation of concerns between observation, cognitive assistance, routing, and execution. The deterministic taxonomy/router is the first-class decision point: it maps incoming telemetry signals, including O-RAN Cloud Notifications envelopes [8], onto harness-defined taxonomy classes and route types. The LLM has no direct write authority over the plant and is not the primary router. It is consulted only when the deterministic router returns the `ambiguous` class.

By forcing all remediation state through strict schemas such as `RCA.json` and `RemediationProposal.json`, the harness converts diagnostic output into a predictable, strongly typed artifact. The agent runtime does not execute commands; it produces candidate structure. The environment does not trust prose, model confidence, or free-form instructions. It accepts only schema-valid fields, deterministic route classes, policy outcomes, and operator decisions.

3 Cognitive Containment via Schemas

When the diagnostic agent evaluates a fault, it is not permitted to generate free-form text or direct API invocations. Instead, it must draft a formal remediation proposal. The structure of this proposal is dictated by a strict JSON Schema definition.

A valid remediation proposal requires several deterministic elements. It must provide a `taxonomyMatch` that is cleanly grounded in the O-RAN Working Group 6

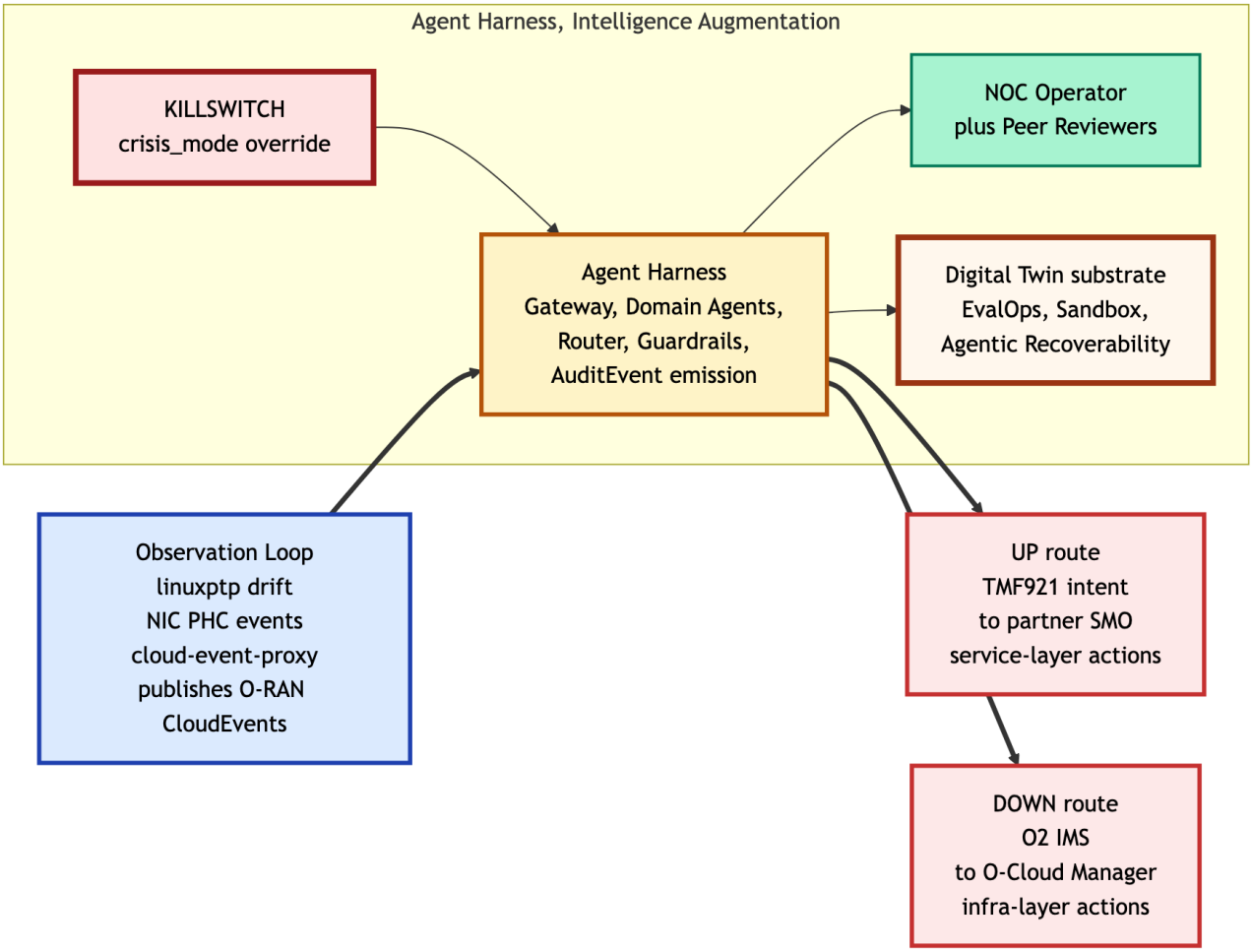

Figure 1: Seven-component macro architecture. The deterministic router and governance layer bound any LLM influence: non-ambiguous cases route without an LLM, ambiguous cases may receive LLM support, and no proposal crosses the execution boundary without validated structure, policy approval, and human co-authorization.

resource model [10]. It must specify an `actionType` (e.g., applying a MachineConfig or executing a bare-metal firmware write) and name the precise `actionTarget` within the cluster topology.

This rigid structure ensures that if the LLM hallucinates an invalid target, invents an unknown taxonomy class, or attempts a malformed action sequence, the resulting payload will fail to conform to the schema. The schema validation engine traps and aborts the hallucination long before it approaches the network's execution boundaries.

4 Schema Gate, Policy Guardrail, and Human Co-Authorization

Before any structural proposal is allowed to leave the remediation boundary, it must pass through three deterministic control points: a schema gate, a five-subcheck policy guardrail, and human co-authorization.

The **schema gate** validates the payload against the defined draft-07 JSON Schema. This step rejects malformed data types, missing required fields, unapproved enumerations, invalid taxonomy classes, and malformed remediation or reversibility-profile subobjects.

The **five-subcheck policy guardrail** is an LLM-free policy enforcement point. Its subchecks fire in order: `crisis_mode` global override, sandbox `apply_allowed`, `action_allowlist`, `blast_radius` caps, and `requiresHumanApproval`. The v0 blast-radius caps are intentionally narrow: one node, one site, and four cells. The v0 sandbox verdict is fixture-driven; production wiring would source the verdict from a mirrored cluster or equivalent simulation environment.

The final control point is **human co-authorization**. The operator receives the TMF688-shaped audit event, the sandbox verdict, the policy result, the optional TMF921-shaped companion intent and dispatch result, and the seven-field ReversibilityProfile before commit. In v0 this is a boolean approval boundary; proposal editing before commit is future work.

After such an `AuditEvent` exists, an LLM-backed chat assistant may summarize or explain the deterministic evidence for the operator. That post-event explanation

does not change the taxonomy match, guardrail result, sandbox verdict, companion-intent outcome, or human approval state.

5 Shape-Correct Standards Handoffs

To maintain trust without overstating maturity, the harness shapes its decisions into established telecom envelopes. These are standards-shaped handoffs, not claims of live standards-body interop. The architecture deliberately avoids proprietary execution channels while acknowledging that v0 uses fixture replay and stubbed boundaries.

TM Forum TMF688 (Event Management API): All human review requests, sandbox verdicts, and governance logs are formatted as TMF688 `AuditEvent` envelopes [14]. This structure embeds a seven-field Reversibility Profile, ensuring the operational evidence chain integrates seamlessly with standard enterprise observability streams.

TM Forum TMF921 (Intent Management API): When an infrastructure action requires service-layer coordination—such as requesting a maintenance window before an intrusive reboot—the harness issues a TMF921-shaped companion intent upward to the partner SMO [9, 15].

O-RAN O2 IMS-shaped boundary: Infrastructure remediation directives are represented as O2 IMS-shaped payloads and routed downward to a v0 O-Cloud Manager stub boundary [11, 12]. The v0 evidence is a shape-correct `o2ims_dispatch` envelope plus CRD-shaped fixtures, not live O2 IMS interop.

6 Execution Delegation to Native Reconcilers

The risk-reducing discipline of the structural communication model is **execution delegation**. The Agent Harness itself is not the live mutation engine for infrastructure.

Once the operator co-authorizes a valid `RemediationProposal`, the v0 O-Cloud Manager stub records the intended O2 IMS-shaped dispatch and points to committed Kubernetes CRD-shaped fixtures. In production, the corresponding execution would be delegated to native reconcilers rather than performed by the harness. Host OS configurations map to the Machine Config Operator (MCO) [13]; out-of-tree driver swaps map to Kernel Module Management (KMM) [3]; and out-of-band firmware updates map to the Metal3 BareMetal Operator (BMO) using DMTF Redfish [1, 6]. The safety claim is not that v0 performs live reconciliation, but that execution authority terminates at structural contracts and trusted reconcilers rather than at LLM output.

7 Discussion

The structural communication pattern described here is highly complementary to orchestration platforms like Nephio [4] and ONAP [5]. While those systems excel at managing the standing topology and lifecycle of network functions, the Agent Harness addresses post-deployment, anomaly-driven Day-2 remediation.

A common criticism of AI-driven automation is the lack of determinism and accountability. The schema gate, five-subcheck policy guardrail, standards-shaped handoffs, and human co-authorization boundary directly address these concerns. The key design point is ordering: deterministic taxonomy routing happens first, and LLM support is limited to ambiguous cases. The human operator remains the execution authority, with the audit event carrying the evidence needed to approve, reject, or defer the proposed change.

The live lab chat path is therefore an explanation surface, not an authority surface. It can read scoped slash-command outputs and a harness-walker sidecar, but it cannot turn prose into a routed or approved production mutation.

8 Honest Scope

The v0 claim is deliberately bounded. The harness demonstrates schema-gated proposal flow, a five-subcheck policy guardrail, human co-authorization, TMF688/TMF921-shaped records, an O2 IMS-shaped stub boundary, and CRD-shaped fixtures. It does not prove live O2 IMS interoperability, live TMF921 exchange, live Redfish writes, live Kubernetes reconciler commits, or standards-body conformance for the harness-authored schemas. Those are v1 interop and validation targets, not v0 claims.

9 Conclusion

Closed-loop remediation in an O-RAN Cloud RAN environment cannot safely rely on autonomous AI acting directly on physical infrastructure. This paper has argued for structural demarcation instead: observe through standards-shaped envelopes, classify through a deterministic taxonomy/router, allow LLM support only for ambiguous cases, and pass every proposed mutation through a schema gate, five-subcheck policy guardrail, and human co-authorization. The v0 harness demonstrates this boundary with fixture replay, an O2 IMS-shaped stub boundary, TMF688/TMF921-shaped records, and CRD-shaped fixtures rather than live interop. This structural communication model provides a safer blueprint for operationalizing generative AI near critical cyber-physical telecom systems without making the model itself an execution authority.

Acknowledgements

This paper builds upon a prior multivendor diagnostic substrate co-authored with S. Kiani Mehr, N.

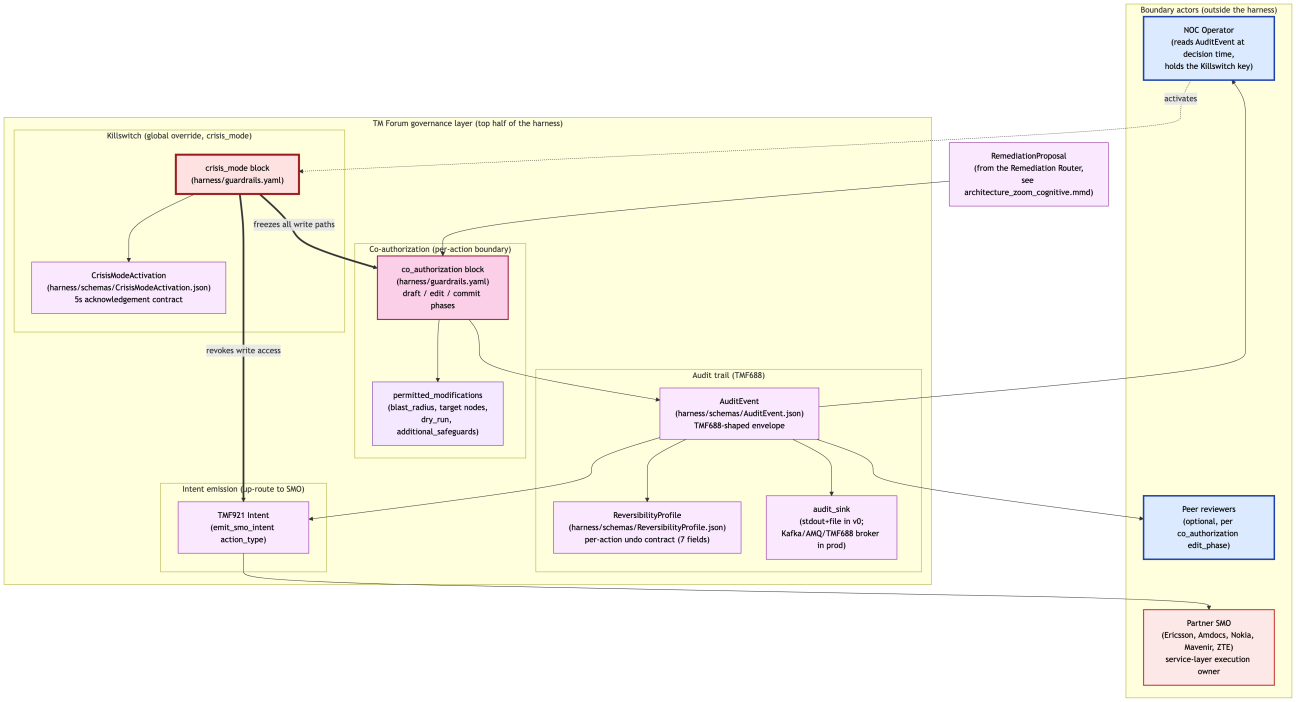

Figure 2: Governance layer zoom. The proposals that survive the twin gates are formatted into standard TMF688 audit envelopes, exposing the reversibility profile and the O2 IMS dispatch structures to the human operator for final authorization.

Korati Prasanna, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh [2]. Source code is Apache-2.0 licensed and available at https://github.com/dkypuros/oran-agent-harness.

References

[1] Distributed Management Task Force. Redfish specification (DMTF DSP0266). https://www.dmtf.org/standards/redfish, 2026. Out-of-band management of servers and infrastructure. Accessed 14 May 2026.

[2] S. Kiani Mehr, N. Korati Prasanna, D. Kypuros, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh. Multivendor agentic AI solution for cloud RAN troubleshooting. Ericsson Technology Blog. https://www.ericsson.com/en/blog/2026/5/multivendor-agentic-ai-solution-for-cloud-ran, 2026. Published 11 May 2026. Accessed 14 May 2026.

[3] Kubernetes Special Interest Groups. Kernel module management operator (KMM). https://github.com/kubernetes-sigs/kernel-module-management, 2026. Accessed 14 May 2026.

[4] Linux Foundation Networking. Nephio: Cloud-native network automation. https://nephio.org, 2026. Accessed 14 May 2026.

[5] Linux Foundation Networking. Open network automation platform (ONAP). https://www.onap.org, 2026. Accessed 14 May 2026.

[6] Metal3 project. Baremetal operator: BareMetalHost, HostFirmwareSettings, HostFirmwareComponents Custom Resource Definitions. https://github.com/metal3-io/baremetal-operator, 2026. Accessed 14 May 2026.

[7] O-RAN Alliance. O-RAN architecture description, working group 1 specification. https://www.o-ran.org/specifications, 2026. Accessed 14 May 2026.

[8] O-RAN Alliance. O-RAN cloud notifications specification, working group 6 (O-RAN.WG6.Cloud-Notifications). https://www.o-ran.org/specifications, 2026. Accessed 14 May 2026.

[9] O-RAN Alliance. O-RAN decoupled SMO architecture, working group 1 technical report (O-RAN.WG1.TR.Decoupled-SMO-Architecture-R004-v03.00). https://www.o-ran.org/specifications, 2026. Accessed 30 May 2026. Source for SMOS terminology.

[10] O-RAN Alliance. O-RAN O2 general aspects and principles, version 01.02 (O-RAN.WG6.O2-GAnP-v01.02). https://www.o-ran.org/specifications, 2026. Working Group 6 specification, with successor in R005

release train (O-RAN.WG6.TS.O2-GA&P-R005-v10.00). Accessed 14 May 2026.

[11] O-RAN Alliance. O-RAN O2 IMS interface specification (infrastructure management services), O-RAN.WG6.O2IMS-Interface. https://www.o-ran.org/specifications, 2026. Working Group 6 specification. Accessed 14 May 2026.

[12] OpenShift KNI. oran-o2ims: O-Cloud manager. https://github.com/openshift-kni/oran-o2ims, 2026. Accessed 14 May 2026.

[13] OpenShift project. Machine config operator: declarative configuration of Red Hat Enterprise Linux CoreOS. https://github.com/openshift/machine-config-operator, 2026. Accessed 14 May 2026.

[14] TM Forum. TMF688 event management API user guide and REST specification. https://www.tmforum.org/oda/open-apis/table/tmf688, 2026. Accessed 14 May 2026.

[15] TM Forum. TMF921 intent management API user guide and REST specification. https://www.tmforum.org/oda/open-apis/table/tmf921, 2026. Accessed 14 May 2026.